
Sistemas Operativos: Simulador Kernel

Maddi Hipólito
18/05/2026

Índice

1. Introducción	3
1.1. Descripción del proyecto	3
1.2. Objetivos del proyecto	3
1.3. Herramientas	3
2. Fundamentos teóricos	4
2.1. Arquitectura de computadores	4
2.2. Procesos y planificación	4
2.3. Memoria virtual	4
2.4. MMU y TLB	5
2.5. Concurrencia y sincronización	5
3. Arquitectura del sistema	5
4. Planificación de procesos	6
4.1. Estructuras de datos del planificador	6
4.2. Política de planificación	7
4.3. Mecanismo de intercambio de colas	8
4.4. Asignación y restauración de contexto	8
4.5. Bucle principal del scheduler	9
5. Gestión de Memoria	10
5.1. Ampliación de la arquitectura	10
5.2. Estructuras de datos del gestor de memoria	10
5.3. Carga de programas en memoria	11
5.4. Traducción de direcciones	12
5.5. Ejecución de instrucciones	13
6. Conclusiones	14

1. Introducción

1.1. Descripción del proyecto

En este proyecto se plantea el desarrollo de un simulador de kernel capaz de reproducir el funcionamiento básico de un sistema operativo. Para ello, se implementan mecanismos de gestión de procesos, planificación, memoria virtual y sincronización, permitiendo simular la ejecución de múltiples procesos sobre una arquitectura compuesta por CPUs, núcleos e hilos hardware.

1.2. Objetivos del proyecto

Los objetivos principales de este proyecto son los siguientes:

- Diseñar e implementar un simulador de kernel utilizando el lenguaje de programación C.
- Modelar una arquitectura hardware compuesta por CPUs, núcleos de procesamiento, hilos hardware y memoria principal.
- Implementar un sistema de planificación de procesos capaz de gestionar múltiples procesos concurrentes mediante colas de prioridad, asignación de procesadores y cambios de contexto.
- Desarrollar un gestor de memoria basado en memoria virtual, incorporando tablas de páginas, una MMU y una memoria TLB para la traducción de direcciones.
- Simular el funcionamiento temporal del sistema mediante componentes como el reloj (*clock*) y el temporizador (*timer*).
- Analizar la interacción entre los distintos subsistemas del kernel y comprender los mecanismos internos utilizados por los sistemas operativos para la gestión eficiente de recursos.
- Aplicar conceptos fundamentales de la asignatura de Sistemas Operativos mediante una implementación práctica en lenguaje C.

1.3. Herramientas

El simulador ha sido desarrollado íntegramente utilizando el lenguaje de programación C, siguiendo el estándar ANSI C con el objetivo de garantizar la portabilidad y compatibilidad del código entre diferentes plataformas.

Para la implementación de la concurrencia y la simulación de los distintos componentes del sistema se ha empleado la biblioteca POSIX Threads (**pthread**s). Gracias a esta biblioteca ha sido posible representar mediante hilos de ejecución independientes elementos como el reloj del sistema, el temporizador, el planificador de procesos y el generador de procesos.

Asimismo, se han utilizado mecanismos de sincronización proporcionados por **pthread**s, tales como mutexes y variables de condición, con el fin de coordinar correctamente el acceso a recursos compartidos y evitar condiciones de carrera durante la ejecución del simulador.

El desarrollo se ha realizado siguiendo una arquitectura modular, organizando el código en diferentes módulos especializados en tareas concretas como la gestión de memoria, la planificación de procesos, la simulación de la arquitectura hardware y la generación de procesos.

2. Fundamentos teóricos

2.1. Arquitectura de computadores

Un sistema informático moderno está compuesto por uno o varios procesadores encargados de ejecutar instrucciones y gestionar los recursos del sistema. Cada procesador puede estar formado por varios núcleos de ejecución (*cores*) y, a su vez, cada núcleo puede disponer de varios hilos hardware capaces de ejecutar diferentes procesos de forma concurrente.

Los sistemas operativos utilizan esta arquitectura para distribuir la carga de trabajo entre los distintos recursos disponibles, maximizando el aprovechamiento del hardware y mejorando el rendimiento global del sistema.

2.2. Procesos y planificación

Un proceso es una instancia de un programa en ejecución. Cada proceso dispone de información propia, como su identificador, estado, registros del procesador y espacio de memoria asignado.

La planificación de procesos (*scheduling*) es una de las funciones principales del kernel. Su objetivo es decidir qué proceso debe ejecutarse en cada instante de tiempo. Para ello, los procesos suelen organizarse en colas de planificación según distintos criterios, como la prioridad o el orden de llegada.

Cuando un proceso deja de ejecutarse para que otro ocupe el procesador, se realiza un cambio de contexto (*context switch*), almacenando el estado del proceso actual y restaurando el del siguiente proceso seleccionado.

2.3. Memoria virtual

La memoria virtual es un mecanismo que permite a cada proceso disponer de un espacio de direcciones independiente del resto de procesos. Para traducir las direcciones virtuales utilizadas por los programas a direcciones físicas de memoria se emplean tablas de páginas.

Cada entrada de la tabla de páginas contiene información sobre la ubicación física de una página de memoria y diferentes bits de control, como los bits de validez, presencia, referencia o modificación.

Este mecanismo proporciona aislamiento entre procesos y facilita una gestión más eficiente de la memoria disponible.

2.4. MMU y TLB

La Unidad de Gestión de Memoria (*Memory Management Unit*, MMU) es el componente encargado de realizar la traducción entre direcciones virtuales y físicas durante la ejecución de un programa.

Para acelerar este proceso, los procesadores incorporan una memoria asociativa denominada TLB (*Translation Lookaside Buffer*), que almacena temporalmente las traducciones más recientes. Cuando una traducción se encuentra en la TLB, el acceso a memoria puede realizarse sin consultar la tabla de páginas, reduciendo significativamente el tiempo de acceso.

2.5. Concurrencia y sincronización

En sistemas con múltiples hilos de ejecución es necesario coordinar el acceso a los recursos compartidos para evitar inconsistencias. Para ello se emplean mecanismos de sincronización como mutexes y variables de condición.

Estos mecanismos permiten proteger estructuras de datos compartidas y coordinar la ejecución de distintos hilos, garantizando el correcto funcionamiento del sistema incluso cuando varias tareas se ejecutan simultáneamente.

3. Arquitectura del sistema

La primera fase del proyecto ha consistido en el diseño e implementación de la arquitectura básica del simulador. El objetivo de esta etapa es definir los distintos componentes del sistema, las estructuras de datos necesarias y los mecanismos de sincronización que permitirían la comunicación entre los diferentes subsistemas.

El simulador ha sido desarrollado utilizando programación concurrente mediante la biblioteca `pthread`, donde cada uno de los componentes principales del sistema se ejecuta como un hilo independiente. La comunicación entre estos hilos se realiza mediante memoria compartida, utilizando mutexes y variables de condición para garantizar la correcta sincronización.

La arquitectura implementada está formada por los siguientes subsistemas:

- **Clock:** representa el reloj principal del sistema y genera los ciclos de ejecución que controlan el avance temporal del simulador.
- **Timer:** recibe los pulsos generados por el reloj y produce interrupciones periódicas (*ticks*) con una frecuencia configurable.
- **Process Generator:** genera nuevos procesos de forma automática y los inserta en las estructuras de planificación.
- **Scheduler/Dispatcher:** encargado de gestionar la planificación y asignación de procesos a los distintos hilos hardware del sistema.

Dado que el simulador está formado por múltiples hilos que comparten estructuras de datos comunes. Para garantizar la sincronización entre los distintos hilos se han utilizado mutexes y variables de condición (pthread mutex y pthread condition variables), evitando condiciones de carrera y permitiendo la coordinación entre los diferentes subsistemas del simulador.

Los mutexes permiten proteger las secciones críticas en las que varios hilos podrían acceder simultáneamente a los mismos recursos compartidos, evitando condiciones de carrera. Las variables de condición se emplean para coordinar la activación de los distintos subsistemas, permitiendo que determinados hilos permanezcan bloqueados hasta recibir una señal de ejecución. Por su parte, los semáforos se utilizan para controlar la sincronización entre eventos y asegurar que las distintas tareas del sistema se ejecuten en el orden adecuado.

Gracias a estos mecanismos se garantiza el correcto funcionamiento del simulador, evitando accesos concurrentes no controlados, pérdidas de información y situaciones de bloqueo derivadas de una sincronización incorrecta entre hilos.

4. Planificación de procesos

Una vez implementada la arquitectura básica del sistema, se ha desarrollado el subsistema de planificación encargado de gestionar la asignación de procesos a los distintos hilos hardware de la máquina simulada.

Para ello, se ha optado por una política de planificación basada en **colas de prioridad con expulsión por prioridad y por quantum**. Esta estrategia permite priorizar la ejecución de determinados procesos sin renunciar a un reparto equilibrado del tiempo de procesador.

4.1. Estructuras de datos del planificador

Los procesos se organizan en una *cola de colas* de tipo P_FCFS, donde cada nivel de prioridad tiene su propia cola `p_queue`. Cada cola mantiene un array de punteros a PCBs junto con punteros al primer y último proceso, permitiendo inserciones y extracciones eficientes.

```
typedef struct p_queue {
    PCB **lista;
    PCB *first;
    PCB *last;
    int size;
    int num_process;
} p_queue;

typedef struct P_FCFS {
    p_queue **colaPrio;
    int size;
    int q;
```

```
    int num_colas;
} P_FCFS;
```

El sistema mantiene dos instancias de esta estructura: `rColaColas` (cola de preparados activa) y `fColaColas` (cola de replanificación), cuyo intercambio se describe más adelante.

4.2. Política de planificación

Los procesos generados por el sistema se han organizado en diferentes colas según su nivel de prioridad. Cada cola almacena procesos con la misma prioridad y sigue un criterio FIFO para resolver el orden de ejecución entre ellos. El Scheduler selecciona siempre procesos pertenecientes a la cola de mayor prioridad disponible.

La política implementada es de tipo expulsora (*preemptive*), por lo que un proceso en ejecución puede ser interrumpido antes de finalizar en dos situaciones:

- **Expulsión por prioridad:** cuando se ha generado o activado un proceso con una prioridad superior a la del proceso que se está ejecutando.
- **Expulsión por quantum:** cuando el proceso ha consumido completamente el tiempo de CPU que le ha sido asignado.

La comprobación de expulsión por prioridad se realiza comparando la prioridad del proceso en ejecución con la del proceso en cabeza de la cola de preparados:

```
static int hay_mayor_prioridad(hilo_t *hilo) {
    if (hilo->r_pcb == NULL) return 0;

    for (int i = 0; i < rColaColas.num_colas; i++) {
        p_queue *queue = rColaColas.colaPrio[i];
        if (queue && queue->num_process != 0) {
            PCB *proceso = queue->first;
            PCB *act      = hilo->r_pcb;
            if (proceso && proceso->prio < act->prio) return 1;
        }
    }
    return 0;
}
```

Cuando se detecta un proceso de mayor prioridad, el proceso en ejecución se mueve a la cola de replanificación `fColaColas` y se realiza el cambio de contexto hacia el nuevo proceso:

```

if (hay_mayor_prioridad(hilo)) {
    PCB *sig = sig_process(&rColaColas);

    if (sig) {
        add_process(hilo->r_pcb, &fColaColas); /* expulsado a replanificacion */
        cambiar_context(hilo, sig);
        core->quantum = QUANTUM;
        core->ejec      = k;
    }
}

```

4.3. Mecanismo de intercambio de colas

Este diseño incluye un mecanismo de intercambio de colas que garantiza que ningún proceso quede indefinidamente sin ejecutarse. Cuando la cola de preparados (`rColaColas`) queda vacía y todos los hilos hardware están ociosos, la cola de replanificación (`fColaColas`) pasa a convertirse en la nueva cola activa, mientras que la antigua se reinicia como nueva cola de replanificación:

```

if (core->ejec == -1 &&
    rColaColas.num_colas == 0 &&
    fColaColas.num_colas != 0) {

    P_FCFS tmp      = rColaColas;
    rColaColas      = fColaColas;
    fColaColas      = tmp;
    restart_politica(&fColaColas);
    printf("Cambiando cola de preparados por cola de finalizados\n");
}

```

4.4. Asignación y restauración de contexto

La función `asig_proc_hilo` asigna el siguiente proceso disponible a un hilo libre. Si el proceso ya había sido ejecutado previamente, se restauran su contador de programa y el contenido de todos sus registros desde el PCB, reanudando la ejecución exactamente donde fue interrumpido:

```

static void asig_proc_hilo(hilo_t *hilo)
{
    if (!hilo || hilo->r_pcb) return;

```

```

PCB *p = sig_process(&r_colaColas);
if (!p) return;

hilo->r_pcb = p;
hilo->estado = 0;
hilo->PTBR   = p->mm.pgb;

if (p->PC != -1) {                               /* proceso previamente ejecutado */
    hilo->PC = p->PC;
    for (int l = 0; l < NUM_REGS; l++)
        hilo->regs[l] = p->regs[l];
}
}

```

4.5. Bucle principal del scheduler

El scheduler se ejecuta como un hilo independiente que permanece bloqueado hasta recibir la señal del timer en cada tick. En cada activación realiza, en orden: eliminación de procesos sin vida, expulsión por prioridad y asignación de procesos a hilos libres.

```

void *scheduler_thread(void *arg)
{
    while (running) {
        pthread_mutex_lock(&clock_mutex);
        pthread_cond_wait(&scheduler_cond, &clock_mutex);

        elim_proc_sin_vida_ejec(); /* eliminar procesos agotados en ejecucion */
        elim_proc_sin_vida_colas(); /* eliminar procesos agotados en colas */
        expulsar_process();         /* expulsion por prioridad */
        asig_process();             /* asignar procesos a hilos libres */

        pthread_mutex_unlock(&clock_mutex);
    }
    return NULL;
}

```

Finalmente, la planificación se ha diseñado de forma modular para permitir futuras ampliaciones, como la incorporación de nuevas políticas de scheduling o la parametrización dinámica del comportamiento del sistema.

5. Gestión de Memoria

Una vez implementadas la arquitectura básica del sistema y la política de planificación, se ha ampliado el simulador incorporando un sistema de gestión de memoria virtual. Esta tercera parte del proyecto ha supuesto una modificación importante de la arquitectura original, ya que los procesos dejan de ser simples estructuras de datos para convertirse en programas reales cargados en memoria y ejecutados por los hilos hardware del sistema.

Con el objetivo de mantener una organización clara del código, se ha añadido un nuevo fichero denominado `process_manager.c` encargado de centralizar las funciones relacionadas con la ejecución de instrucciones, la gestión del estado de los procesos y las operaciones auxiliares necesarias durante la simulación. Esta separación ha permitido reducir el tamaño y la complejidad de otros módulos del proyecto, mejorando su legibilidad y facilitando el mantenimiento del código.

5.1. Ampliación de la arquitectura

La incorporación de la memoria virtual ha requerido modificar varios componentes previamente desarrollados. En primer lugar, el antiguo *Process Generator* ha sido sustituido por un *Loader*, encargado de cargar programas almacenados en memoria secundaria simulada mediante ficheros `.elf`. Cada programa contiene dos segmentos: un segmento de código (`.text`) y un segmento de datos (`.data`).

Asimismo, cada hilo hardware dispone ahora de nuevos elementos propios de una arquitectura con memoria virtual:

- Un contador de programa (PC) que indica la siguiente instrucción a ejecutar.
- Un banco de registros de propósito general.
- Una Unidad de Gestión de Memoria (MMU).
- Una memoria asociativa o TLB (*Translation Lookaside Buffer*).
- Un registro PTBR (*Page Table Base Register*) que apunta a la tabla de páginas del proceso actualmente asignado.

Por otro lado, se ha implementado una memoria física dividida en marcos de página (*frames*), reservando una zona específica para el Kernel donde se almacenan las tablas de páginas de todos los procesos.

5.2. Estructuras de datos del gestor de memoria

Para implementar el sistema de memoria virtual se han definido varias estructuras de datos encargadas de representar tanto la memoria física como las tablas de páginas de los procesos.

La memoria física simulada tiene un tamaño total de 64 MB y se encuentra dividida en marcos de página (*frames*) de tamaño fijo. Además, una parte inicial de la memoria queda reservada para el Kernel, donde se almacenan las tablas de páginas de todos los procesos.

Cada entrada de una tabla de páginas se representa mediante la estructura `page_t`, que contiene el identificador del marco físico asociado y un bit de validez que indica si la página está correctamente mapeada.

```
typedef struct {  
    int frame_id;  
    int valida;  
} page_t;
```

Las tablas de páginas de cada proceso se representan mediante la estructura `page_table_t`, que almacena el conjunto de páginas del proceso y el número total de entradas.

```
typedef struct {  
    page_t *pages;  
    int num_pages;  
    int id;  
} page_table_t;
```

Por otro lado, la memoria física mantiene información sobre cada marco mediante la estructura `frame_t`, indicando si está libre, qué proceso lo ocupa y qué página contiene.

```
typedef struct {  
    int libre;  
    int pid;  
    int pagina;  
} frame_t;
```

Finalmente, se han definido dos estructuras globales para representar la memoria virtual y la memoria física del sistema. Estas estructuras permiten gestionar las tablas de páginas existentes, controlar los marcos libres y realizar las traducciones de direcciones necesarias durante la ejecución de los programas.

5.3. Carga de programas en memoria

El subsistema *Loader* sustituye al generador de procesos utilizado en las fases anteriores. En lugar de crear procesos de forma aleatoria, se encarga de leer programas almacenados en ficheros `.elf`, reservar memoria física para ellos y crear las estructuras necesarias para su ejecución.

Para cada programa se crea una tabla de páginas, se reservan los marcos físicos necesarios y se copia el contenido de los segmentos de código y datos a memoria física. Finalmente, la información correspondiente se almacena en el PCB del proceso.

El siguiente fragmento muestra la creación de la tabla de páginas y su registro en el sistema de memoria virtual:

```
page_table_t *tabla = crear_tabla(pag_tot, frames, proceso->pid);

int pgb = add_ptable(tabla);

proceso->mm.code = val_text;
proceso->mm.data = val_data;
proceso->mm.pgb = pgb;
```

5.4. Traducción de direcciones

Una vez cargado un programa, todas las referencias a memoria se realizan mediante direcciones virtuales. Para acceder a memoria física es necesario traducir dichas direcciones utilizando la tabla de páginas asociada al proceso.

La dirección virtual se divide en dos partes: un número de página virtual (VPN) y un desplazamiento dentro de la página (*offset*). A partir del VPN se obtiene el marco físico correspondiente y se construye la dirección física final.

```
int vpn = hilo->PC / TAM_PAGE;
int offset = hilo->PC % TAM_PAGE;

int frame = tabla->pages[vpn].frame_id;
int pa = frame * TAM_PAGE + offset;
```

Además, cada hilo hardware dispone de una TLB que almacena traducciones recientes. En esta implementación la TLB únicamente almacena entradas correspondientes a accesos a datos. Las instrucciones no se almacenan en la TLB debido a que normalmente se ejecutan de forma secuencial y no se reutilizan con la misma frecuencia que los datos, por lo que mantenerlas en la caché de traducción supondría un uso poco eficiente del espacio disponible.

A continuación, se detalla el flujo secuencial que realiza la MMU para completar la traducción de la dirección lógica a la física:

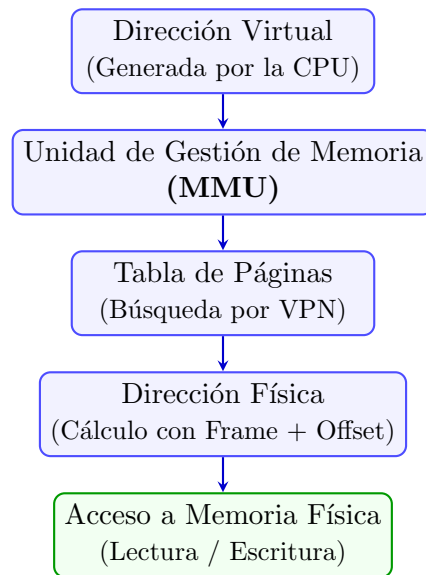


Figura 1: Flujo del proceso de traducción de direcciones en el simulador.

5.5. Ejecución de instrucciones

Cada hilo hardware ejecuta las instrucciones almacenadas en memoria física siguiendo el valor del contador de programa (PC). El conjunto de instrucciones implementado es reducido y está compuesto por las operaciones LD, ST, ADD y EXIT.

La instrucción ADD suma el contenido de dos registros y almacena el resultado en un tercero:

```
case 0x2:
{
    reg = (inst >> 24) & 0xF;
    reg1 = (inst >> 20) & 0xF;
    reg2 = (inst >> 16) & 0xF;

    hilo->regs[reg] = hilo->regs[reg1] + hilo->regs[reg2];
    break;
}
```

Cuando se produce un cambio de contexto, el estado del proceso se conserva almacenando el valor del contador de programa y el contenido de todos los registros en el PCB. Asimismo, la TLB asociada al hilo se reinicia para evitar reutilizar traducciones pertenecientes al proceso anterior. Cuando dicho proceso vuelva a ejecutarse, toda esta información será restaurada automáticamente.

6. Conclusiones

A lo largo de este proyecto se ha desarrollado un simulador de kernel capaz de reproducir algunos de los mecanismos fundamentales presentes en un sistema operativo moderno. El trabajo se ha estructurado en tres etapas principales: la construcción de la arquitectura básica del sistema, la implementación de un planificador de procesos y el desarrollo de un sistema de memoria virtual.

En la primera fase se diseñó una arquitectura multihilo basada en `pthread`s, compuesta por los subsistemas *Clock*, *Timer*, *Loader* y *Scheduler/Dispatcher*. Posteriormente se incorporó una política de planificación basada en prioridades y expulsión por quantum, permitiendo simular cambios de contexto y la gestión de múltiples procesos de forma concurrente.

Finalmente, se implementó un sistema simplificado de memoria virtual que incluye tablas de páginas, memoria física paginada, traducción de direcciones mediante MMU y una TLB asociada a cada hilo hardware. Además, se añadieron mecanismos de carga de programas, ejecución de instrucciones y conservación del contexto de los procesos.

El desarrollo de este proyecto ha permitido profundizar en conceptos fundamentales de los sistemas operativos, como la planificación de procesos, la sincronización entre hilos, la gestión de memoria y la ejecución concurrente. Asimismo, ha servido para comprender la complejidad que supone coordinar múltiples subsistemas que interactúan simultáneamente dentro de un kernel.

7. Limitaciones y trabajo pendiente

A pesar de que la mayor parte de las funcionalidades previstas han sido implementadas correctamente, no se ha conseguido completar completamente la fase final del proyecto.

Durante las pruebas finales del simulador se ha detectado un error de tipo *Segmentation Fault* que aparece de forma sistemática cuando comienza la ejecución del proceso con identificador 19. El comportamiento observado sugiere la existencia de algún acceso inválido a memoria relacionado con la gestión de procesos, las estructuras de planificación o el sistema de memoria virtual.

Se han realizado diversas tareas de depuración para localizar el origen del problema, revisando especialmente las operaciones de cambio de contexto, la liberación de memoria de los PCBs y el acceso a las tablas de páginas. Sin embargo, debido a la complejidad de las interacciones entre los distintos subsistemas y a las limitaciones de tiempo disponibles para el desarrollo del proyecto, no ha sido posible identificar y corregir completamente el error antes de la entrega.

Como trabajo futuro, sería necesario continuar el proceso de depuración utilizando herramientas específicas como `gdb` o `valgrind`, con el objetivo de localizar el acceso inválido que provoca el fallo y garantizar la estabilidad del simulador para un número arbitrario de procesos.